

DIA 6.1 — Preparando o Banco de Dados para Resolver os Exercícios SQL

Introdução

Antes de começar os desafios da Aula 6, precisamos preparar um banco de dados com informações suficientes para praticar consultas simples, filtros, ordenações, funções de agregação, relacionamentos, agrupamentos e subconsultas.

Neste capítulo, criaremos um pequeno banco de dados para representar uma operação comercial.

A estrutura será composta por três tabelas:

- `clientes`;
- `produtos`;
- `pedidos`.

A tabela `clientes` armazenará os dados das pessoas cadastradas.

A tabela `produtos` conterá os itens disponíveis para venda, incluindo preço e quantidade em estoque.

Já a tabela `pedidos` será responsável por registrar quais produtos foram comprados por cada cliente e em qual quantidade.

Essa estrutura será utilizada nos exercícios e nas resoluções apresentados nos próximos capítulos.

Visão Geral do Banco de Dados

O relacionamento entre as tabelas pode ser representado da seguinte maneira:

```
CLIENTES
-----
cod_cliente
nome_cliente
sobrenome_cliente

      1
      |
      | realiza
      |
      N

PEDIDOS
-----
cod_pedido
```

```
cod_cliente  
cod_produto  
qtde
```

```
      N  
      |  
      | contém  
      |  
      1
```

```
PRODUTOS  
-----  
cod_produto  
nome_produto  
descricao  
preco  
qtde_estoque
```

Um cliente pode realizar vários pedidos.

Um produto também pode aparecer em vários pedidos.

A tabela `pedidos` é responsável por conectar clientes e produtos.

Criando a Tabela de Clientes

Começaremos pela tabela `clientes`.

```
CREATE TABLE clientes (  
  cod_cliente SERIAL PRIMARY KEY,  
  nome_cliente VARCHAR(50) NOT NULL,  
  sobrenome_cliente VARCHAR(50) NOT NULL  
);
```

Entendendo a estrutura

A coluna:

```
cod_cliente SERIAL PRIMARY KEY
```

representa o identificador único de cada cliente.

O tipo `SERIAL` faz com que o PostgreSQL gere os códigos automaticamente de forma sequencial.

A restrição `PRIMARY KEY` garante que:

- Cada cliente possua um código único;
- O código não possa ser nulo;
- Nenhum código seja repetido.

As colunas:

```
nome_cliente VARCHAR(50) NOT NULL,  
sobrenome_cliente VARCHAR(50) NOT NULL
```

armazenam o nome e o sobrenome do cliente.

A restrição `NOT NULL` determina que essas informações sejam obrigatórias.

Criando a Tabela de Produtos

Agora criaremos a tabela responsável pelo catálogo de produtos.

```
CREATE TABLE produtos (  
    cod_produto SERIAL PRIMARY KEY,  
    nome_produto VARCHAR(50) NOT NULL,  
    descricao TEXT,  
    preco NUMERIC(10,2) CHECK (preco > 0),  
    qtde_estoque SMALLINT DEFAULT 0  
);
```

Código do produto

A coluna:

```
cod_produto SERIAL PRIMARY KEY
```

é o identificador único do produto.

Assim como na tabela de clientes, o PostgreSQL gerará automaticamente seus valores.

Nome e descrição

A coluna:

```
nome_produto VARCHAR(50) NOT NULL
```

armazena o nome do produto e não permite valores nulos.

Já a coluna:

```
descricao TEXT
```

armazena uma descrição mais detalhada.

Como não possui `NOT NULL`, a descrição é opcional.

Isso permitirá, por exemplo, cadastrar um produto sem preencher esse campo.

Preço

A coluna:

```
preco NUMERIC(10,2)
```

armazena valores com até dez dígitos, sendo dois deles destinados às casas decimais.

A restrição:

```
CHECK (preco > 0)
```

impede o cadastro de preços iguais ou inferiores a zero.

Por exemplo, esta tentativa seria rejeitada:

```
INSERT INTO produtos (  
    nome_produto,  
    preco  
)  
VALUES (  
    'Produto inválido',  
    -10.00  
);
```

Quantidade em estoque

A coluna:

```
qtde_estoque SMALLINT DEFAULT 0
```

armazena a quantidade disponível em estoque.

O `DEFAULT 0` determina que, caso nenhuma quantidade seja informada, o banco utilizará automaticamente o valor zero.

Criando a Tabela de Pedidos

A terceira tabela armazenará os pedidos realizados.

```
CREATE TABLE pedidos (  
  cod_pedido SERIAL PRIMARY KEY,  
  cod_cliente INT REFERENCES clientes(cod_cliente),  
  cod_produto INT REFERENCES produtos(cod_produto),  
  qtde SMALLINT NOT NULL  
);
```

Código do pedido

A coluna:

```
cod_pedido SERIAL PRIMARY KEY
```

identifica cada pedido de forma única.

Relacionamento com clientes

A coluna:

```
cod_cliente INT REFERENCES clientes(cod_cliente)
```

é uma chave estrangeira.

Ela conecta cada pedido a um registro existente na tabela `clientes`.

Isso significa que não será possível cadastrar um pedido para um cliente inexistente.

Por exemplo, caso não exista um cliente de código 99, o banco rejeitará:

```
INSERT INTO pedidos (  
    cod_cliente,  
    cod_produto,  
    qtde  
)  
VALUES (  
    99,  
    1,  
    2  
);
```

Relacionamento com produtos

A coluna:

```
cod_produto INT REFERENCES produtos(cod_produto)
```

conecta o pedido a um produto existente.

Assim, o banco também impede que um pedido faça referência a um produto que não esteja cadastrado.

Quantidade comprada

A coluna:

```
qtde SMALLINT NOT NULL
```

armazena a quantidade comprada.

Como possui `NOT NULL`, esse valor é obrigatório.

Para melhorar ainda mais a integridade do banco, poderíamos adicionar uma restrição para permitir apenas quantidades positivas:

```
qtde SMALLINT NOT NULL CHECK (qtde > 0)
```

Neste capítulo, utilizaremos essa versão mais segura na estrutura completa.

Inserindo os Clientes

Agora cadastraremos cinco clientes.

```
INSERT INTO clientes (  
    nome_cliente,  
    sobrenome_cliente  
)  
VALUES  
    ('João', 'Silva'),  
    ('Maria', 'Oliveira'),  
    ('Pedro', 'Santos'),  
    ('Ana', 'Carvalho'),  
    ('Lucas', 'Moraes');
```

Ao utilizar `SERIAL`, não precisamos informar manualmente o código.

O PostgreSQL gerará os seguintes identificadores:

```
1 – João Silva  
2 – Maria Oliveira  
3 – Pedro Santos  
4 – Ana Carvalho  
5 – Lucas Moraes
```

Esses códigos serão utilizados posteriormente na tabela de pedidos.

Inserindo os Produtos

Agora cadastraremos os produtos utilizados nos exercícios.

```
INSERT INTO produtos (  
    nome_produto,  
    descricao,  
    preco,  
    qtde_estoque  
)  
VALUES  
    (  
        'Pizza Margherita',  
        'Pizza tradicional com molho de tomate e manjeriço',  
        45.00,  
        10  
    ),  
    (  
        'Pizza Margherita',  
        'Pizza tradicional com molho de tomate e manjeriço',  
        45.00,  
        10  
    ),  
    (  
        'Pizza Margherita',  
        'Pizza tradicional com molho de tomate e manjeriço',  
        45.00,  
        10  
    ),  
    (  
        'Pizza Margherita',  
        'Pizza tradicional com molho de tomate e manjeriço',  
        45.00,  
        10  
    ),  
    (  
        'Pizza Margherita',  
        'Pizza tradicional com molho de tomate e manjeriço',  
        45.00,  
        10  
    );
```

```

        'Refrigerante Lata',
        'Lata de 350 ml',
        6.00,
        50
    ),
    (
        'Hambúrguer',
        'Hambúrguer artesanal',
        30.00,
        20
    ),
    (
        'Batata Frita',
        'Porção de batata frita',
        18.00,
        15
    ),
    (
        'Cerveja',
        'Long Neck',
        8.00,
        25
    ),
    (
        'Detergente',
        'Detergente líquido de 500 ml',
        1.89,
        32
    ),
    (
        'Leite Integral',
        'Leite integral em caixa de 1 litro',
        6.90,
        70
    ),
    (
        'Refrigerante',
        'Garrafa de refrigerante de 600 ml',
        4.10,
        14
    ),
    (
        'Refrigerante',
        'Garrafa de refrigerante de 1 litro',
        50.00,
        16
    ),
    (
        'Refrigerante',
        'Lata de 350 ml',
        2.99,

```



```

        45
    ),
    (
        'Suco Integral',
        'Suco integral em caixa',
        7.00,
        100
    ),
    (
        'Margarina',
        NULL,
        11.00,
        70
    );

```

Não informamos manualmente a coluna `cod_produto`.

Como ela utiliza `SERIAL`, os códigos serão gerados automaticamente de 1 a 12.

Essa abordagem evita problemas com a sequência interna do PostgreSQL.

Por que não informar manualmente o código de uma coluna SERIAL?

É possível fazer algo como:

```

INSERT INTO produtos (
    cod_produto,
    nome_produto,
    preco
)
VALUES (
    6,
    'Detergente',
    1.89
);

```

Entretanto, ao informar manualmente o valor da coluna `SERIAL`, a sequência interna pode não ser atualizada.

Imagine que o PostgreSQL tenha gerado automaticamente os códigos de 1 a 5.

Se inserirmos manualmente os códigos de 6 a 12, a sequência poderá continuar acreditando que o próximo valor disponível é 6.

Em uma inserção futura, o banco tentaria gerar novamente o código 6 e produziria um erro de chave duplicada.

Por esse motivo, quando uma coluna utiliza `SERIAL`, o mais seguro é permitir que o próprio banco gere o valor.

Caso seja realmente necessário informar códigos manualmente, a sequência deverá ser ajustada posteriormente com `setval`.

Inserindo os Pedidos

Agora registraremos algumas compras.

```
INSERT INTO pedidos (  
    cod_cliente,  
    cod_produto,  
    qtde  
)  
VALUES  
    (1, 1, 2),  
    (2, 3, 1),  
    (1, 2, 3),  
    (3, 5, 4),  
    (4, 4, 1);
```

Esses registros representam as seguintes operações:

```
João Silva comprou:  
- 2 Pizzas Margherita;  
- 3 Refrigerantes em lata.
```

```
Maria Oliveira comprou:  
- 1 Hambúrguer.
```

```
Pedro Santos comprou:  
- 4 Cervejas.
```

```
Ana Carvalho comprou:  
- 1 porção de Batata Frita.
```

O cliente Lucas Moraes foi cadastrado, mas ainda não realizou nenhum pedido.

Essa diferença será útil para exercícios envolvendo clientes sem pedidos.

Analizando os Pedidos Inseridos

A primeira linha:

```
(1, 1, 2)
```

significa:

```
cod_cliente = 1  
cod_produto = 1  
qtde = 2
```

Portanto:

```
João Silva comprou duas Pizzas Margherita.
```

A linha:

```
(2, 3, 1)
```

indica:

```
Maria Oliveira comprou um Hambúrguer.
```

Observe que a tabela de pedidos não armazena o nome do cliente nem o nome do produto.

Ela armazena somente seus códigos.

Para recuperar as descrições completas, utilizaremos `JOIN`.

Script Completo

A seguir está o script completo, pronto para ser executado no PostgreSQL.

```
DROP TABLE IF EXISTS pedidos;  
DROP TABLE IF EXISTS produtos;  
DROP TABLE IF EXISTS clientes;  
  
CREATE TABLE clientes (  
    cod_cliente SERIAL PRIMARY KEY,  
    nome_cliente VARCHAR(50) NOT NULL,
```

```

        sobrenome_cliente VARCHAR(50) NOT NULL
    );

CREATE TABLE produtos (
    cod_produto SERIAL PRIMARY KEY,
    nome_produto VARCHAR(50) NOT NULL,
    descricao TEXT,
    preco NUMERIC(10,2) NOT NULL CHECK (preco > 0),
    qtde_estoque SMALLINT NOT NULL DEFAULT 0 CHECK (qtde_estoque >= 0)
);

CREATE TABLE pedidos (
    cod_pedido SERIAL PRIMARY KEY,
    cod_cliente INT NOT NULL,
    cod_produto INT NOT NULL,
    qtde SMALLINT NOT NULL CHECK (qtde > 0),

    CONSTRAINT fk_pedidos_clientes
        FOREIGN KEY (cod_cliente)
        REFERENCES clientes(cod_cliente),

    CONSTRAINT fk_pedidos_produtos
        FOREIGN KEY (cod_produto)
        REFERENCES produtos(cod_produto)
);

INSERT INTO clientes (
    nome_cliente,
    sobrenome_cliente
)
VALUES
    ('João', 'Silva'),
    ('Maria', 'Oliveira'),
    ('Pedro', 'Santos'),
    ('Ana', 'Carvalho'),
    ('Lucas', 'Moraes');

INSERT INTO produtos (
    nome_produto,
    descricao,
    preco,
    qtde_estoque
)
VALUES
    (
        'Pizza Margherita',
        'Pizza tradicional com molho de tomate e manjeriç o',
        45.00,
        10
    ),
    (

```

```

    'Refrigerante Lata',
    'Lata de 350 ml',
    6.00,
    50
),
(
    'Hambúrguer',
    'Hambúrguer artesanal',
    30.00,
    20
),
(
    'Batata Frita',
    'Porção de batata frita',
    18.00,
    15
),
(
    'Cerveja',
    'Long Neck',
    8.00,
    25
),
(
    'Detergente',
    'Detergente líquido de 500 ml',
    1.89,
    32
),
(
    'Leite Integral',
    'Leite integral em caixa de 1 litro',
    6.90,
    70
),
(
    'Refrigerante',
    'Garrafa de refrigerante de 600 ml',
    4.10,
    14
),
(
    'Refrigerante',
    'Garrafa de refrigerante de 1 litro',
    50.00,
    16
),
(
    'Refrigerante',
    'Lata de 350 ml',
    2.99,

```

```

        45
    ),
    (
        'Suco Integral',
        'Suco integral em caixa',
        7.00,
        100
    ),
    (
        'Margarina',
        NULL,
        11.00,
        70
    );

INSERT INTO pedidos (
    cod_cliente,
    cod_produto,
    qtde
)
VALUES
    (1, 1, 2),
    (2, 3, 1),
    (1, 2, 3),
    (3, 5, 4),
    (4, 4, 1);

```

Por que utilizar DROP TABLE IF EXISTS?

No início do script, utilizamos:

```

DROP TABLE IF EXISTS pedidos;
DROP TABLE IF EXISTS produtos;
DROP TABLE IF EXISTS clientes;

```

Esses comandos removem as tabelas caso elas já existam.

Isso permite executar todo o script novamente sem receber mensagens informando que as tabelas já foram criadas.

A ordem é importante.

Primeiro removemos `pedidos`, porque ela possui chaves estrangeiras apontando para `clientes` e `produtos`.

Somente depois removemos as tabelas referenciadas.

Atenção: `DROP TABLE` apaga a tabela e todos os dados armazenados nela. Esse comando deve ser utilizado com cuidado em bancos de dados reais.

Conferindo os Dados Inseridos

Depois de executar o script, podemos verificar os clientes:

```
SELECT *  
FROM clientes  
ORDER BY cod_cliente;
```

Podemos conferir os produtos:

```
SELECT *  
FROM produtos  
ORDER BY cod_produto;
```

E também visualizar os pedidos:

```
SELECT *  
FROM pedidos  
ORDER BY cod_pedido;
```

Conferindo os Relacionamentos

Para visualizar os pedidos com o nome do cliente e do produto, podemos utilizar:

```
SELECT  
    pe.cod_pedido,  
    c.nome_cliente,  
    c.sobrenome_cliente,  
    pr.nome_produto,  
    pe.qtde,  
    pr.preco,  
    pr.preco * pe.qtde AS valor_item  
FROM pedidos pe  
JOIN clientes c  
    ON c.cod_cliente = pe.cod_cliente  
JOIN produtos pr  
    ON pr.cod_produto = pe.cod_produto  
ORDER BY pe.cod_pedido;
```

Essa consulta permite verificar se os relacionamentos foram cadastrados corretamente.

O resultado apresentará:

```
pedido  
cliente  
produto  
quantidade  
preço  
valor do item
```

O valor do item é calculado por meio da expressão:

```
pr.preco * pe.qtde
```

Informações Importantes para os Exercícios

A base foi preparada para permitir diferentes tipos de análise.

Clientes com pedidos

Os seguintes clientes possuem compras cadastradas:

```
João Silva  
Maria Oliveira  
Pedro Santos  
Ana Carvalho
```

Cliente sem pedido

O cliente:

```
Lucas Moraes
```

não possui pedidos.

Esse cenário pode ser utilizado em exercícios com `LEFT JOIN`, `NOT EXISTS` e subconsultas.

Produtos vendidos

Os produtos que aparecem em pedidos são:

Pizza Margherita
Refrigerante Lata
Hambúrguer
Batata Frita
Cerveja

Produtos não vendidos

Os demais produtos foram cadastrados, mas ainda não possuem pedidos.

Esse cenário permite resolver exercícios envolvendo:

- Produtos vendidos;
- Produtos nunca vendidos;
- Produtos com pedidos;
- Produtos sem pedidos.

Diferentes preços e estoques

A tabela contém produtos:

- Com preço maior que R\$ 20,00;
- Com preço inferior a R\$ 20,00;
- Com estoque menor que 20 unidades;
- Com estoque igual a 20 unidades;
- Com estoque superior a 20 unidades;
- Com descrição nula;
- Com nomes repetidos, como `Refrigerante`.

Essas variações permitem praticar:

WHERE
BETWEEN
ORDER BY
LIMIT
COUNT
AVG
MAX
MIN
DISTINCT
IS NULL

Exercícios que Podem ser Resolvidos com essa Base

Com essa estrutura, será possível responder perguntas como:

- Quais clientes estão cadastrados?
- Quais produtos custam mais de R\$ 20,00?
- Quais são os produtos mais baratos?
- Quais produtos possuem estoque baixo?
- Quem comprou determinado produto?
- Quanto cada cliente gastou?
- Qual é o valor total das vendas?
- Qual cliente gastou mais?
- Quais produtos já foram vendidos?
- Quais produtos nunca foram vendidos?
- Quais clientes ainda não realizaram pedidos?
- Qual é o preço médio dos produtos?

Essa pequena estrutura já permite praticar grande parte dos fundamentos da linguagem SQL.

Conclusão

Neste capítulo preparamos o banco de dados que será utilizado para resolver os exercícios da Aula 6.

Criamos três tabelas relacionadas:

```
clientes
produtos
pedidos
```

Também cadastramos clientes, produtos e pedidos com diferentes características, permitindo a prática de consultas simples e avançadas.

A partir desta base, será possível trabalhar com:

```
SELECT
WHERE
ORDER BY
LIMIT
JOIN
DISTINCT
COUNT
SUM
AVG
MAX
```

```
MIN  
GROUP BY  
HAVING  
IN  
EXISTS
```

Com o ambiente preparado, o próximo passo é começar a resolver os desafios e compreender como cada necessidade pode ser transformada em uma consulta SQL.